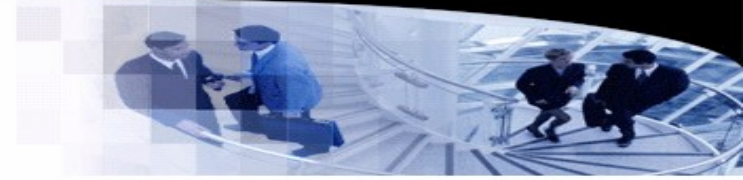




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

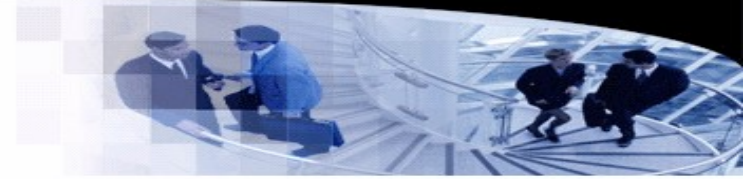
Lecture 9. Simple Types

袁梦霆 (YUAN Mengting)

ymt@whu.edu.cn

School of Computer Science, Wuhan University

Types



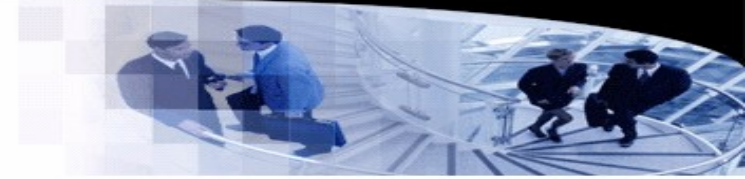
- **Stuckness**

- Recall the language of arithmetic expressions we have designed in the last lecture. Terms like “`succ false`” cannot be evaluated, which means they are stuck.
- Stuckness represents runtime errors.
- However, some of these errors can be identified at compile-time (statically), instead of runtime.

- **Types**

- To avoid such errors, we classify terms into **types**.
- For the language of arithmetic expressions, we introduce two types, `Nat` and `Bool`.
- “A term `t` has type `T`” means that `t` “obviously” evaluates to a value in `T`.
- By “obviously” we mean it can be determined **statically**, without actually running the evaluation.

Types



- Typing relation

- The typing relation $t : T$ (or $t \in T$) is defined by a set of rules assigning terms to types.
- Typing rules for booleans:

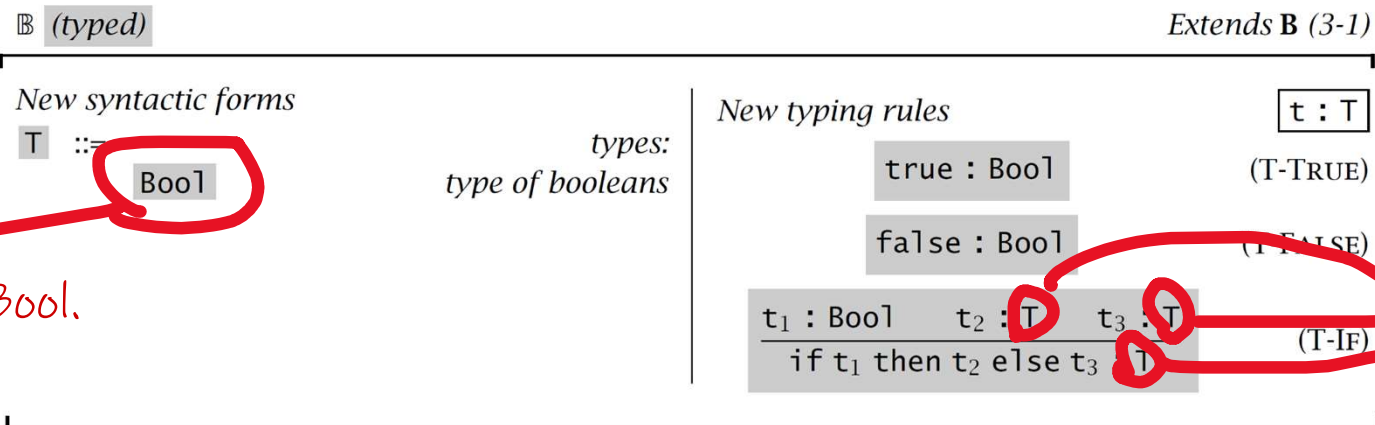
$\mathbb{B}$ (typed)		Extends $\mathbf{B}$ (3-1)	
New syntactic forms		New typing rules	
$T ::=$			$t : T$
Bool	types: type of booleans	$\text{true} : \text{Bool}$	(T-TRUE)
		$\text{false} : \text{Bool}$	(T-FALSE)
		$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T}$	(T-IF)

- T-TRUE and T-FALSE assign the type Bool to boolean constants.
- T-IF assigns a type to a conditional expression, when the premises are satisfied.

Types

- Typing relation

- The typing relation $t : T$ (or $t \in T$) is defined by a set of rules assigning terms to types.
- Typing rules for booleans:



A new type, Bool.

Metavariables

- T-TRUE and T-FALSE assign the type `Bool` to boolean constants.
- T-IF assigns a type to a conditional expression, when the premises are satisfied.

Types

- Typing relation

- Typing rules for numbers:

$\mathbb{B} \approx$ (typed)

Extends NB (3-2) and 8-1

New syntactic forms

$T ::= \dots$

Nat

types:
type of natural numbers

New typing rules

0 : Nat

$t : T$

(T-ZERO)

$t_1 : \text{Nat}$
 $\text{succ } t_1 : \text{Nat}$

(T-SUCC)

$t_1 : \text{Nat}$
 $\text{pred } t_1 : \text{Nat}$

(T-PRED)

$t_1 : \text{Nat}$
 $\text{iszero } t_1 : \text{Bool}$

(T-ISZERO)

Types

- Typing relation

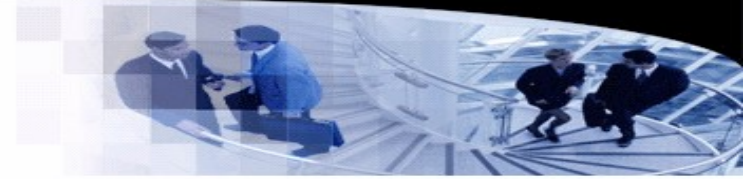
- Typing rules for numbers:

If the type of t_1 is Nat , then the type of $\text{succ } t_1$ is Nat .

$\mathbb{B} \approx \mathbb{N}$ (typed)		Extends NB (3-2) and 8-1	
<i>New syntactic forms</i>			
$T ::= \dots$	<i>types:</i>		
Nat	<i>type of natural numbers</i>	$\frac{t_1 : \text{Nat}}{\text{succ } t_1 : \text{Nat}}$	(T-SUCC)
<i>New typing rules</i>		$\frac{t_1 : \text{Nat}}{\text{pred } t_1 : \text{Nat}}$	(T-PRED)
$0 : \text{Nat}$	$t : T$ (T-ZERO)	$\frac{t_1 : \text{Nat}}{\text{iszero } t_1 : \text{Bool}}$	(T-ISZERO)

If the type of t_1 is Nat , then the type of $\text{iszero } t_1$ is Bool .

Types

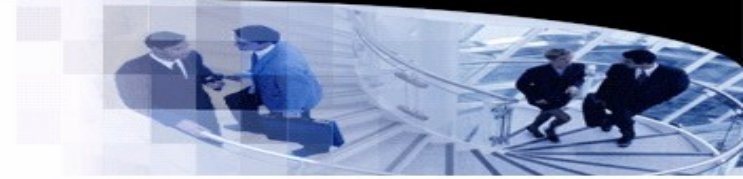


- Typing relation

- The typing relation for arithmetic expressions is the smallest binary relation between terms and types satisfying all instances of the typing rules.
- A term t is **typable** (or **well typed**) if there is some T such that $t : T$.
- Example: show that “if iszero 0 then 0 else pred 0 : Nat”.

$$\frac{\frac{\frac{}{0 : \text{Nat}} \text{T-ZERO}}{\text{iszero } 0 : \text{Bool}} \text{T-ISZERO} \quad \frac{}{0 : \text{Nat}} \text{T-ZERO} \quad \frac{\frac{}{0 : \text{Nat}} \text{T-ZERO}}{\text{pred } 0 : \text{Nat}} \text{T-PRED}}{\text{if iszero 0 then 0 else pred 0 : Nat}} \text{T-IF}$$

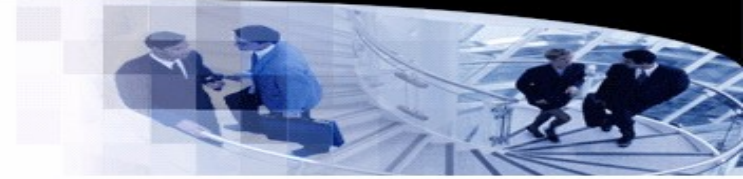
Types



- **Typing relation**

- Now that we have designed the typing rules for the language. Are these rules “good”?
- **Lemma** [Inversion of the typing relation]:
 - If $\text{true} : R$, then $R = \text{Bool}$.
 - If $\text{false} : R$, then $R = \text{Bool}$.
 - If $\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : R$, then $t_1 : \text{Bool}$, $t_2 : R$, and $t_3 : R$.
 - If $0 : R$, then $R = \text{Nat}$.
 - If $\text{succ } t_1 : R$, then $R = \text{Nat}$ and $t_1 : \text{Nat}$.
 - If $\text{pred } t_1 : R$, then $R = \text{Nat}$ and $t_1 : \text{Nat}$.
 - If $\text{iszero } t_1 : R$, then $R = \text{Bool}$ and $t_1 : \text{Nat}$.
 - ■
- **Theorem** [Uniqueness of Types]: Each term t has at most one type.
 - Proof: Straightforward structural induction on t , using the appropriate clause of the inversion lemma (plus the induction hypothesis) for each case.

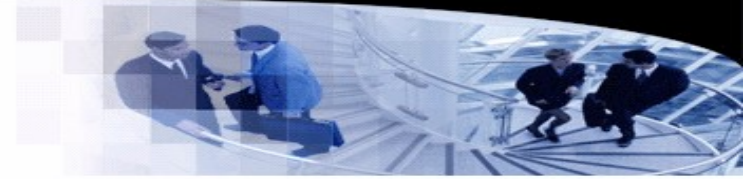
Safety



- **Safety = Progress + Preservation**

- Safety (soundness) is the most basic property of a type system.
- Safety means that well-typed terms do not “go wrong”.
- Progress:
 - A well-typed term is not stuck.
- Preservation:
 - If a well-typed term takes a step of evaluation, then the resulting term is also well typed.
- These properties together tell us that a well-typed term can never reach a stuck state during evaluation.
- **Lemma** [Canonical Forms]:
 - 1. If v is a value of type `Bool`, then v is either `true` or `false`.
 - 2. If v is a value of type `Nat`, then v is a numeric value according to the grammar.

Safety



- Progress

- **Theorem** [Progress]: Suppose t is a well-typed term (that is, $t : T$ for some T). Then either t is a value or else there is some t' with $t \rightarrow t'$.

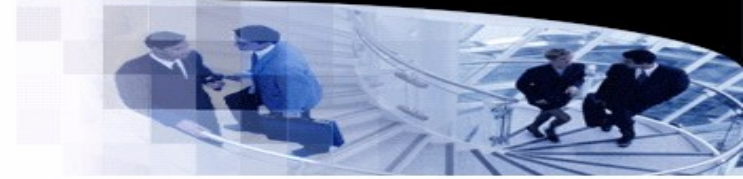
- Proof:

- By induction on a derivation of $t : T$.
- The T-TRUE, T-FALSE, and T-ZERO cases are immediate, since t in these cases is a value.
- Consider case T-IF, where $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$.
- By the induction hypothesis, either t_1 is a value or else there is some t_1' such that $t_1 \rightarrow t_1'$.
- If t_1 is a value, then the canonical forms lemma assures us that it must be either true or false, in which case either E-IFTRUE or E-IFFALSE applies to t .
- If $t_1 \rightarrow t_1'$, then by E-IF, $t \rightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3$.
- Other cases can be proved similarly.
- ■

$$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \quad (\text{T-IF})$$

$$\frac{t_1 \rightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3} \quad (\text{E-IF})$$

Safety



• Preservation

- **Theorem** [Preservation]: If $t : T$ and $t \rightarrow t'$, then $t' : T$.

- Proof:

- By induction on a derivation of $t : T$.
- Consider case T-If, where $t = \text{if } t_1 \text{ then } t_2 \text{ else } t_3$.
- By the premises, we know that $t_1 : \text{Bool}$, $t_2 : T$, and $t_3 : T$.
- There are three rules by which $t \rightarrow t'$ can be derived.

$$\frac{t_1 : \text{Bool} \quad t_2 : T \quad t_3 : T}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 : T} \quad (\text{T-If})$$

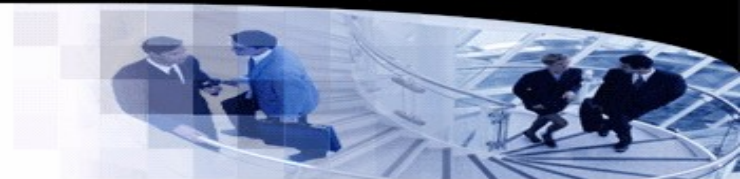
- E-IfTrue: t evaluates to t_2 , whose type is T .
 $\text{if true then } t_2 \text{ else } t_3 \rightarrow t_2 \quad (\text{E-IfTrue})$
- E-IfFalse: Similar to E-IfTrue.
- E-If: $t' = \text{if } t_1' \text{ then } t_2 \text{ else } t_3$. Since $t_1 : \text{Bool}$, we know $t_1' : \text{Bool}$ by induction hypothesis. Then we can apply rule T-If to conclude that $t' : T$.

- Other cases can be proved similarly.

- ■

$$\frac{t_1 \rightarrow t_1'}{\text{if } t_1 \text{ then } t_2 \text{ else } t_3 \rightarrow \text{if } t_1' \text{ then } t_2 \text{ else } t_3} \quad (\text{E-If})$$

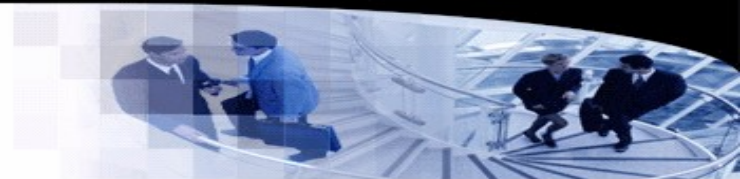
Simply Typed Lambda-Calculus



- Function types

- The type constructor $\rightarrow$ is used to represent a function type.
- $T_1 \rightarrow T_2$ is the set of all functions from T_1 to T_2 , which is the type of functions from T_1 to T_2 .
- $\rightarrow$ is right associate.
- The expression $T_1 \rightarrow T_2 \rightarrow T_3$ stands for $T_1 \rightarrow (T_2 \rightarrow T_3)$.
- Examples:
 - $\text{Bool} \rightarrow \text{Bool}$
 - $(\text{Bool} \rightarrow \text{Bool}) \rightarrow (\text{Bool} \rightarrow \text{Bool})$

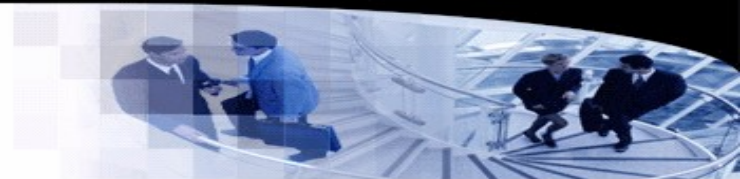
Simply Typed Lambda-Calculus



- Typing relation

- In order to assign a type to $\lambda x . t$, we must know the type of x .
- In this case, the abstraction is written in $\lambda x : T1 . t2$.
- $T1$ is called the **type annotation**.
- Languages in which type annotations in terms are used to help guide the type checker are called **explicitly typed**.
- Languages in which we ask the type checker to infer or reconstruct this information are called **implicitly typed**.

Simply Typed Lambda-Calculus



- Typing relation

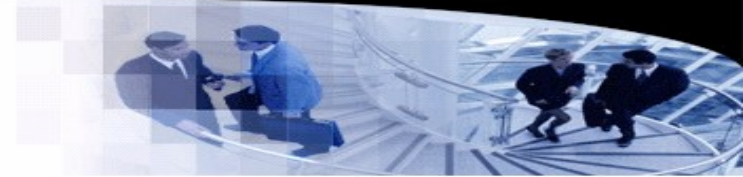
- Once the argument is typed, it is possible to infer the type of the abstraction.

$$\frac{x:T_1 \vdash t_2 : T_2}{\vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$$

- The problem is that t_2 may contain free variables once λx .
- As these many binders may have been removed to expose t_2 , t_2 may have many kinds of free variables.
- Thus, we need to know the types of all free variables to infer the type of t_2 .
- These type assignments to free variables are wrapped into a context set Γ .
- By adding the context Γ , the typing rule can be written as follows.

$$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$$

Simply Typed Lambda-Calculus



- Typing relation

- Pure simply typed lambda-calculus.

$\rightarrow$ (typed)

Based on λ (5-3)

Syntax

$t ::=$

x

$\lambda x:T. t$

$t t$

$v ::=$

$\lambda x:T. t$

$T ::=$

$T \rightarrow T$

$\Gamma ::=$

$\emptyset$

$\Gamma, x:T$

terms:

variable

abstraction

application

values:

abstraction value

types:

type of functions

contexts:

empty context

term variable binding

Evaluation

$t \rightarrow t'$

$t_1 \rightarrow t'_1$

$t_1 t_2 \rightarrow t'_1 t_2$

(E-APP1)

$t_2 \rightarrow t'_2$

$v_1 t_2 \rightarrow v_1 t'_2$

(E-APP2)

$(\lambda x:T_{11}. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12}$ (E-APPABS)

Typing

$\Gamma \vdash t : T$

$x:T \in \Gamma$

$\Gamma \vdash x : T$

(T-VAR)

$\Gamma, x:T_1 \vdash t_2 : T_2$

$\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2$

(T-ABS)

$\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}$

$\Gamma \vdash t_1 t_2 : T_{12}$

(T-APP)

Simply Typed Lambda-Calculus

- Typing relation

- Pure simply typed lambda-calculus.

If $x:T$ is in the context Γ , then the free variable $x:T$ by the context Γ .

Type annotation.

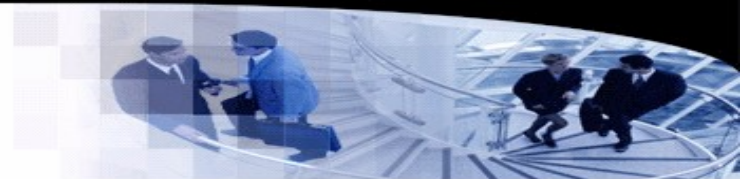
Function type.

The context is a group of type assignments.

$\rightarrow$ (typed)		Based on λ (5-3)	
Syntax		Evaluation	$t \rightarrow t'$
$t ::=$			
x	terms: variable	$\frac{t_1 \rightarrow t'_1}{t_1 t_2 \rightarrow t'_1 t_2}$	(E-APP1)
$\lambda x:T. t$	abstraction	$\frac{t_2 \rightarrow t'_2}{v_1 t_2 \rightarrow v_1 t'_2}$	(E-APP2)
$t t$	application	$(\lambda x:T_{11}. t_{12}) v_2 \rightarrow [x \mapsto v_2] t_{12}$	(E-APPABS)
$v ::=$	values:	Typing	$\Gamma \vdash t : T$
$\lambda x:T. t$	abstraction value	$\frac{x:T \in \Gamma}{\Gamma \vdash x : T}$	(T-VAR)
$T ::=$	types:	$\frac{\Gamma, x:T_1 \vdash t_2 : T_2}{\Gamma \vdash \lambda x:T_1. t_2 : T_1 \rightarrow T_2}$	(T-ABS)
$T \rightarrow T$	type of functions	$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_{11}}{\Gamma \vdash t_1 t_2 : T_{12}}$	(T-APP)
$\Gamma ::=$	contexts:		
$\emptyset$	empty context		
$\Gamma, x:T$	term variable binding		

The type of the result term after function application.

Simply Typed Lambda-Calculus

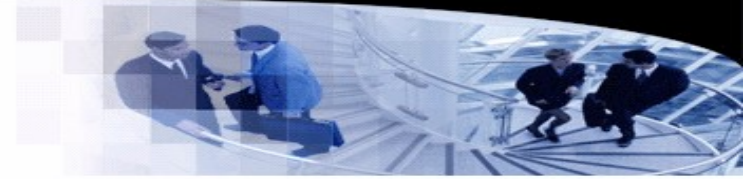


- Typing relation

- Example: Infer the type of $(\lambda x:\text{Bool}. x) \text{ true}$.

$$\frac{\frac{\frac{x:\text{Bool} \in x:\text{Bool}}{x:\text{Bool} \vdash x : \text{Bool}} \text{T-VAR}}{\vdash \lambda x:\text{Bool}. x : \text{Bool} \rightarrow \text{Bool}} \text{T-ABS} \quad \frac{}{\vdash \text{true} : \text{Bool}} \text{T-TRUE}}{\vdash (\lambda x:\text{Bool}. x) \text{ true} : \text{Bool}} \text{T-APP}$$

Simply Typed Lambda-Calculus



- Typing relation

- Example: Infer the type of $(\lambda x:\text{Bool}. x) \text{ true}$.

The context
 $\Gamma = \{x:\text{Bool}\}.$

$$\frac{\frac{\frac{x:\text{Bool} \in x:\text{Bool}}{x:\text{Bool} \vdash x : \text{Bool}} \text{ T-VAR}}{\vdash \lambda x:\text{Bool}. x : \text{Bool} \rightarrow \text{Bool}} \text{ T-ABS} \quad \frac{}{\vdash \text{true} : \text{Bool}} \text{ T-TRUE}}{\vdash (\lambda x:\text{Bool}. x) \text{ true} : \text{Bool}} \text{ T-APP}$$

No context.